Code Assessment
of the Diamond PAU Deploy Smart Contracts

PUBLIC

Date June 10, 2026

Produced for Sky

By CHAINSECURITY

Contents

1 Executive Summary

Dear all,

Thank you for trusting us to help Sky with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Diamond PAU Deploy according to **Scope** to support you in forming an opinion on their security risks.

Sky implements deployment scripts for Diamond PAU Facets and their wirings.

The most critical subjects covered in our audit are functional correctness and access control. The general subjects covered are trustworthiness.

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Diamond PAU Deploy repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	4 June 2026	90df5687155df6ba8ca9b9bcfdf947ff69895405	v1.0.0

For the Solidity smart contracts, the compiler version `0.8.34` was chosen.

The following files were in scope of this assessment:

```
script/
├── DeployFacetsAndWire.s.sol
├── interfaces/
│   └── ExternalFacetInterfaces.sol
└── mainnet/
    └── DeployFacetsAndWireMainnet.s.sol
```

Additionally, note that a deployment validation has been performed as part of the scope, see Deployment Validation.

2.1.1 Excluded from scope

All files not listed above were excluded from this assessment. In particular, the `diamond-pau` dependency under `lib/` — the facet implementations, the `Beacon`, the `Controller`, and the `ALMProxy` — is assumed to be correct and was not reviewed. Third-party libraries and the test suite were likewise out of scope.

3 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Sky provides the deployment scripts for `diamond-pau`, which deploy the integration facets and register them with the `Beacon` so that the `Controller` can route calls to them. This assessment covers these deployment scripts.

`DeployFacetsAndWire` is an abstract Foundry script that performs the deployment in a single broadcast. For each facet it exposes a `_deployAndWire*` helper that deploys the facet and calls `beacon.setIntegration(id, { facet, wires })`, where each wire maps an external (prefixed) selector to the corresponding facet selector (for all facet selectors except `DEFAULT_ADMIN_ROLE()` and `ALLOCATOR_ROLE()`). Its `run` flow loads a per-deployment config (`admin`, `beacon`), calls the abstract `_deployAndWireFacets` function to wire all facets, and finally transfers the `Beacon` admin role from the deployer to the configured `admin`, requiring the two to differ. Which facets are wired is therefore left to a concrete subclass that implements `_deployAndWireFacets`.

`DeployFacetsAndWireMainnet` is the concrete mainnet runner. It forks mainnet, supplies the facet constructor addresses from the Spark and Grove address registries (with the Uniswap V3/V4 and Permit2 addresses hardcoded as constants), selects the environment-specific config file (`staging` / `production`) via the `ENV` variable, and implements `_deployAndWireFacets` to deploy and wire the 25 facets: Aave, Basin, CCTP, Centrifuge, Curve, DAIUSDS, ERC4626, ERC7540, Ethena, Farm, LayerZero, Maple, Merkl, OTC, Pendle, PSM, SparkVault, Superstate, TransferAsset, UniswapV3, UniswapV4, USDS, WEETH, WrapProxyETH, and WSTETH.

4 Trust Model

Due to the scope being scripts, there is no trust model. However, a deployment validation has been performed to validate the deployer's actions, see Deployment Validation.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 `PSM3Facet` Is Not Deployed by the in-Scope Scripts

System Consideration	Version 1

The `diamond-pau` dependency ships a `PSM3Facet`, but the deployment code does not use it. `DeployFacetsAndWire` has no `_deployAndWirePSM3Facet` helper, and `ExternalFacetInterfaces.sol` declares no `IPSM3FacetExternal`. `PSM3Facet` is therefore neither deployed nor wired.

This is expected. `PSM3Facet` integrates the Spark PSM3, which exists on L2s rather than mainnet, where the `PSMFacet` is used instead. The only concrete runner in scope is `DeployFacetsAndWireMainnet`, so omitting PSM3 is correct for this deployment.

SC2 Deployment Validation

System Consideration	Version 1

The contracts are deployed and configured by a potentially untrusted party. Thus, one cannot rely on the deployment scripts being executed honestly or at all. A deployment validation (DV) has been performed. Please see our DV files for more details.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

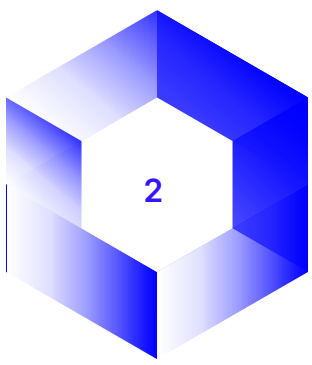
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	**Impact**		
	High	Medium	Low
High	**Critical**	**High**	**Medium**
Medium	**High**	**Medium**	**Low**
Low	**Medium**	**Low**	**Low**

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

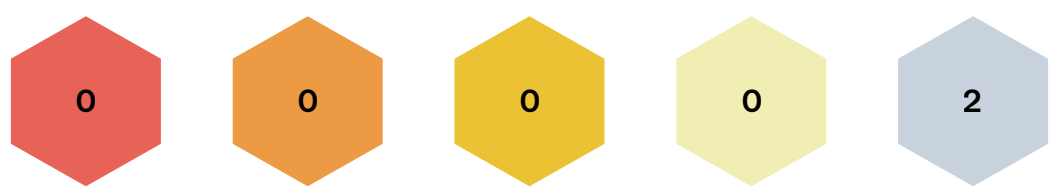
7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



Open Issues

Total number of findings



Critical | High | Medium | Low | Informational

7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Info	#001	`layerZero_quoteTransfer` Is Declared `nonpayable` While the Facet Function Is `view`	Acknowledged
Info	#002	Inconsistent Parameter and Return Value Names Between External Interfaces and Facets	Acknowledged

7.2 Descriptions

#001 `layerZero_quoteTransfer` Is Declared `nonpayable` While the Facet Function Is `view`

Informational | **Version 1** | **Acknowledged**

`ILayerZeroFacetExternal.layerZero_quoteTransfer` is declared without a state mutability modifier, whereas the facet function it is wired to, `LayerZeroFacet.quoteTransfer`, is declared `view`.

State mutability is not part of a function's selector, so the wiring and routing are unaffected and the function behaves identically when called. The discrepancy is only observable through the external interface.

> **Acknowledged**
>
> Sky acknowledged the finding. However, it was preferred not to change the codebase to not require a redeployment.

#002 Inconsistent Parameter and Return Value Names Between External Interfaces and Facets

Informational | **Version 1** | **Acknowledged**

Each facet is wired into the `Beacon` through a prefixed external interface (`I<Facet>External`), whose function selectors form the call side of every wire. While the selectors and types of these external functions match their facet counterparts, several of them name a parameter or return value differently from the facet function they are wired to:

1. `ICCTPFacetExternal.cctp_transfer` names its first parameter `usdcAmount`, whereas `CCTPFacet.transfer` names it `amount`.
2. `IEthenaFacetExternal.ethena_cooldownAssets` names its return value `cooldownShares`, whereas `EthenaFacet.cooldownAssets` names it `shares`.
3. `IEthenaFacetExternal.ethena_cooldownShares` names its return value `cooldownAssets`, whereas `EthenaFacet.cooldownShares` names it `assets`.
4. `IOTCFacetExternal.otc_setBuffer` names its second parameter `otcBuffer`, whereas `OTCFacet.setBuffer` names it `buffer`.
5. `IPendleFacetExternal.pendle_redeem` and `pendle_getRedeemRateLimitKey` name their first parameter `pendleMarket`, whereas `PendleFacet.redeem` and `getRedeemRateLimitKey` name it `market`.
6. `IUSDSFacetExternal.usds_setVault` names its first parameter `vault`, whereas `USDSFacet.setVault` names it `vault_`. Here the wired interface `IUSDSFacet.setVault` also uses `vault`, so the divergence is only with the implementation.
7. `IWEETHFacetExternal.weeth_claimWithdrawal` names its return value `ethReceived`, whereas `WEETHFacet.claimWithdrawal` names it `wethReceived`. Here the wired interface `IWEETHFacet.claimWithdrawal` also uses `ethReceived`, so the divergence is only with the implementation.

Parameter and return value names are not part of a function's selector (which is derived from the name and parameter types only) and do not affect ABI encoding, so the wiring and runtime behavior are unaffected.

Acknowledged

Sky acknowledged the finding. However, it was preferred not to change the codebase to not require a redeployment.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.